

Project 2 – An infectious disease

The model

The infectious disease MS2V-2020 is spreading across the world.* The disease is highly contagious: Any person that catches MS2V-2020 will, after an incubation time of 5 days, become infectious and transmit the disease to anyone they come into close contact with. After 7 more days, the disease subsides and the person is no longer infectious, but they remain immune until 60 days after the original infection. After that, they can catch the illness again. While MS2V-2020 does not lead to death, it can cause considerable discomfort for infected persons. It is therefore imperative to understand how the disease spreads in the community.

The aim of this project is to produce a computer simulation for the spreading of MS2V-2020. We model this disease as follows: Persons are simulated by “agents” that are located on a “chess board” of 50×50 squares. A day in the real world is modelled by one “round” of the simulation, and each of these rounds proceeds as follows (in this order):

- Step 1 – Agents become infected: If any square of the board contains an infectious agent, then all other agents in the same square become infected in this round (unless they are already infected with the disease, or they are currently immune).
- Step 2 – Agents move: Each agent moves by one square on the board, in a direction that will be described below.
- Step 3 – Counters updated: At this point, the time step is reflected in the agent’s data, e.g., for an immune agent, the remaining time of immunity is decreased by 1 round.

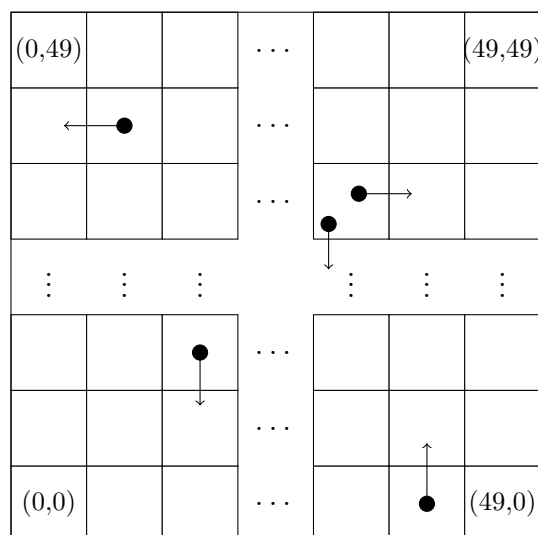


Figure 1: A sketch of the “chessboard” that agents move on. Coordinates given as (x, y) ; agents marked as black dots, arrows indicating direction of movement.

*Health warning: This disease is, of course, fictitious, and any similarity with past, present or future real-life diseases is entirely accidental. We will be programming a “cartoon version” of a so-called *agent-based model* for disease spreading, but none of the predictions here should be taken as realistic for any purpose.

To illustrate the counting of rounds: Suppose that an agent becomes infected in round 10 of the simulation. Then this agent will be infectious from round 15 to round 22 inclusive, but will no longer be infected (no longer carry the disease) after round 22 (i.e., in round 23). The agent will be immune against further infection until round 70 but not in round 71.

The movement of agents on the board is as follows. Positions on the board are labelled with integer coordinates (x, y) , as indicated in Fig. 1. Each agent has a fixed direction of movement – left, right, up, or down – that is assigned at the beginning of the simulation; and each round they move by 1 square in that direction. However, when they have reached the boundary of the board, then rather than moving one step “outside” the board, they reverse direction and take one step in the new direction. They persist with the new direction of movement until they reach the next boundary.

Tasks

Throughout this project, all floating point numbers are of `double` type and all integers of `int` type. All parameters and array entries can be assumed to be non-null without further mention. Any other assumptions on input parameters should be documented in the Javadoc comments. Your code does not need to handle exceptional input values unless this is explicitly specified in the relevant question.

For full marks, make sure that you re-use work from previous question parts (via function/procedure calls) as appropriate.

1) Individual agents

(a) Create a composite data type[†] **Agent** which contains the following fields, all of integer type:

- **x** and **y**, to contain the agent’s current position on the board,
- **direction**, the agent’s current direction of movement, where the values 0, 1, 2, 3 stand for left, right, down, up respectively,
- **timeAfterInfection**, which contains: the number of rounds after the agent has been infected; 0 if the agent has become infected in the current round; -1 if the agent has never been infected, or is no longer immune after an infection.

(b) In the class **AgentActions**, implement the following functions, all of which take an **Agent** record as their only parameter, and return a true/false value. These functions should *not* modify the contents of the input record.

function name	output
isInfected	whether the agent is currently infected with the disease
isInfectable	whether the agent can catch an infection during the current round (rather than being immune)
isInfectious	whether the agent can infect others in the current round

(c) In the same class **AgentActions**, implement the following procedures that take an **Agent** record as their only parameter, and update it as described below.

[†]Note for those with some previous knowledge: Please *do not* add any constructors to the class **Agent**, or indeed any other methods; *do not* declare the visibility of any field to be private. While these techniques may be valid and useful for the model at hand, we will learn about them only in the Spring term, and they will create problems in marking the current project (and may hence result in loss of marks).

function name	aspect/field to be updated
move	The agent moves one step on the board.
timeStep	“Time after infection” reflects that one round has passed.
infect	The agent becomes infected with the disease in the current round.

2) Several agents

The functions/procedures described below should be implemented in the class **Simulator**; they work with an array of agent records (i.e., of type **Agent[]**) as their parameters and/or return values.

- Write a function **countInfected** that takes an array of agent records as input, and returns the number of infected agents in the array.
- Write a function **randomAgents** that takes an integer n (assumed positive) and a real number $0 \leq p \leq 1$ as its input, and returns n agents with randomly chosen parameters as follows:
 - The position of agents is independent and uniformly distributed across the squares.
 - Their direction of movement is also chosen independently and with equal probability from the 4 available directions (left, right, down, up).
 - Each agent becomes infected (independently) with probability p , and infected agents are at the very beginning of the disease period (“infected in the current round”).

Hint: A random integer j , uniformly distributed over $0 \leq j < n$, can be obtained with `(int) floor(n*random())`, supposing that functions of the class `java.lang.Math` have been imported in the usual way. Alternatively, the class `java.util.Random` can be used (cf. its documentation).

3) The simulation

For the core of the simulation, add the following to the class **Simulator**.

- Write a procedure **oneRound**, acting on an array of agent records as its only parameter, that processes one round of the simulation. That is, it performs step (1)–(3) as described on the first page of the assignment sheet, updating the agent records accordingly.
- Write a procedure **runSimulation** which takes four parameters: an integer n , a number $0 \leq p \leq 1$, an integer r , and a file name (as **String**), which is used as the name of the output file described below.[‡] It should run the simulation with n agents, initially distributed at random and with probability p of initial infection (see part 2(b) above), lasting r rounds. The procedure should write the results of the simulation to a text file as follows. For each round, one line should be written with the following integer values, separated by commas:
 - Number of the round (starting from 0),
 - number of infected agents after the round,
 - number of infectious agents after the round.
 - number of agents after the round that are immune, but no longer infected.

I/O related exceptions should not be caught, but rather specified (and documented).

[‡]Please use the file name as given in the parameter and do not, e.g., add an extra file extension.

4) Documentation

Within the source code, add Javadoc comments to every class, every function and procedure and every field (of composite data types). In these, describe the purpose of the function, class or field, and document all parameters and return values. Also, document any assumption about the parameter ranges, and any exceptions thrown.

How to prepare your submission

Start by downloading the template files from Moodle. Most of the Java classes that you will work with are already defined there.

Read the assignment sheet carefully. Be sure to implement all functions and procedures with exactly the names, parameters and return types that are specified, and with parameters in the order as given.

While preparing your code, please follow these style and formatting guidelines. (This forms part of the assessment.)

- Place all code block delimiters – { and } – on separate lines.
- Within code blocks, indent the code by 4 spaces with respect to the surrounding code.
- Choose all names for variables, parameters, functions, and fields to start with a lowercase character.

Make sure that your code compiles correctly. If the source code files that you submit do not compile (for whatever reason), you will receive zero marks for the affected parts of the project.

The code template also includes a unit test. This test does *not* check the output of your code – it only verifies whether you have declared your functions with the correct names and with the correct parameter/return types. You are advised to use the unit test to check your function declarations for any typos.

Test every piece of your code with meaningful test data (including any special or exceptional values). Verify that the output of your functions is plausible.

Once you are satisfied with the code, create a JAR file in BlueJ as follows:

- Select “Project → Create Jar file” in the menu.
- Leave “Main class” set to “none”.
- ***Be sure to tick “Include source”*** and “Include BlueJ project files”.
- Click “Continue” and save the file in a convenient location.

This JAR file is the one that you need to submit.

Please do not mention your name or student number in the code, the documentation, or in file names, as your submission will be graded anonymously.

Hand-in, late submissions

Please submit your work by uploading it on Moodle before

Wednesday, 20 January (week 2 Spring term), 14:00:00h.

Your submission should consist of the JAR file (see above) and of nothing else. Please be sure to press the “Submit assignment” button on Moodle before the deadline. You will receive an e-mail notification from Moodle when your assignment is submitted; if you have not received this, please assume that your submission is not yet complete.

Late submissions will incur a penalty according to the University’s standard rules: “Work which is up to one hour late will have five percent of marks deducted. After one hour, ten percent of the available marks will be deducted for each day (or part of day) that the work is late, up to a total of five days, including weekends and bank holidays, e.g., if work is awarded a mark of 30 out of 50, and the work is up to one day late, the final mark is 25. After five days,

the work is marked zero. Note, however, that the penalty cannot result in a mark less than zero.”

If your submission is affected by Exceptional Circumstances (such as, being ill), you must hand in an Exceptional Circumstances claim, accompanied by evidence. This should be done at least two days *before* the deadline if feasible, and 7 days after the deadline at the very latest. For questions in this respect, contact the Assessment Administrators at maths-exams@york.ac.uk.

Marking

Your submission will be marked as follows. Partial solutions are acceptable, and will be awarded partial marks.

Code correctness (33 marks) Your code will be tested by an automated process (a collection of unit tests) to check whether it works correctly for various values of the input parameters. For each of these test cases that you pass, one mark will be awarded. Marks are awarded entirely on the criterion whether the output values of your code match the conditions specified on this assignment sheet.

Coding style (10 marks) Your source code will be read by the examiner. Marks will be awarded to reflect whether your code is readable, is appropriately structured, and follows the coding style conventions mentioned above. Marks may be subtracted for code that is functional but is implemented in a non-transparent or overly complicated way.

Source code documentation (5 marks) These marks are awarded for appropriate Javadoc comments in the code (see part 4).

In total, 48 marks are available. Your raw mark out of 48 will be moderated and scaled to the University scale 0–100. This scaled mark will contribute 25% towards your final mark for the module.

Academic integrity

To this project, the established Academic Integrity rules apply, as set out by the University. In particular, you must not copy program code or the text of the documentation from fellow students, or from other public or non-public sources. You must also not make your solution available to fellow students before the deadline.

You are allowed to re-use example code from the lectures or practicals (though, where you use it, you should include a comment to that effect in your source).